

A TEARDOWN OF APPLE’S ON-DEVICE FOUNDATION MODELS: ARCHITECTURE, QUANTIZATION, AND CONTAINER FORMATS OF THE FM_GENERATIVEMODELS ASSET SET

YU YAO-HSING

ABSTRACT. We present a static teardown of Apple’s on-device generative-model asset bundle, `com.apple.MobileAsset.UAF.FM.GenerativeModels`, as shipped to a macOS 26/27 machine, together with an attempt to reconstruct a running forward pass from the recovered tensors. The catalogue holds 124 signed assets across six model families; we document the container stack (Cryptex images, the BBBB weight container, the `odix` executable, the ANE `.hwx` binary, the MPSGraph package), the `odix` reference scheme, and two language backbones—a dense ~ 3 B model and a sparse Mixture-of-Experts with instruction-following pruning—plus ~ 90 task adapters, guardrail models, draft models and server variants.

The weight codec is 4-bit LUT palettization with a per-block fp16 scale under an ANE tile permutation. We reverse it, close the parameter budget exactly against the physical file (9.862 B parameters, 100.00% of the shipped weight bytes), and validate the decoder by round-trip against known weights at correlation 0.981—the 4-bit quantization floor. Two obstacles previously believed fatal dissolve: the router $\rightarrow$ expert map is mathematically irrelevant to the output because the pruned feed-forward is an ungated permutation-invariant sum, and the “missing constant table” has a shipped runtime equivalent. A later OS build additionally ships the 3B token embedding that earlier builds withheld.

A coherent from-weights forward pass nevertheless does not follow. We localize the obstruction to a single ANE storage convention—the coefficient ordering used by residual-writing projections—which no available compiler reproduces for an isolated operator. We also retract four of our own intermediate results, each traceable to a null hypothesis that destroyed more structure than the hypothesis under test; we consider that failure mode the most transferable finding in this work. The analysis is read-only and no signing material was bypassed; no weights or tokenizer data are reproduced here or published.

EXECUTIVE SUMMARY

This report documents a static teardown of the on-device model assets that ship with Apple Intelligence, together with an attempt to reconstruct a running forward pass from the recovered tensors. It is written to be read in either direction: as a description of the shipped artefacts (§10–§7), or as a record of which reconstruction claims survived scrutiny and which did not (§8).

What the assets contain. The `UAF.FM.GenerativeModels` catalogue holds 124 signed assets spanning six model families. The weight codec is 4-bit LUT palettization with a per-block fp16 scale, stored under an Apple Neural Engine tile permutation. Both are reversible: the parameter budget closes *exactly* against the physical file (9.862 B parameters, 100.00% of the shipped weight bytes, zero non-finite tensors), and the decoder is validated by round-trip against known weights at correlation 0.981—the 4-bit quantization floor.

What that means in practice. The shipped assets are sufficient to recover the model weights, the architecture, the tokenizer, the router, and—as of macOS 27.0 build 26A5378n—the 3B token embedding, which earlier builds did not ship. Two obstacles that were previously believed fatal turned out not to be: the instruction-following-pruning router $\rightarrow$ expert map is *mathematically*

Date: Revision of July 25, 2026. Asset build 13M204130; catalogue re-checked on macOS 27.0 build 26A5378n.

Key words and phrases. Apple Intelligence, on-device language models, Mixture-of-Experts, weight quantization, Apple Neural Engine, model container formats, reverse engineering.

irrelevant to the output (Finding 9), and the “missing constant table” has a shipped runtime equivalent (Finding 10).

What still does not work. A coherent from-weights forward pass. The remaining gap is narrow and identified: the ANE emits residual-writing projections under a coefficient ordering (OutTrans=1) that no available compiler reproduces for an isolated operator (Finding 25). Five of seven weight roles per layer are decoded correctly; two are not.

On negative results. Four instruments used during this work produced confident-looking conclusions that later proved unfounded, each because the null hypothesis destroyed more structure than the hypothesis under test. Those retractions are recorded in place rather than removed (Findings 20, 23, and §8.4), because the failure mode is the most transferable result here.

Findings at a glance.

#	Finding	Status
1	Weight codec is 4-bit LUT palettization $\times$ per-block fp16 scale	Established
2	Parameter budget closes exactly against the physical file	Established
3	Codec is necessary, then sufficient, for reconstruction	Established
4	Budget closure bounds the search fan-out	Established
5	The ANE spatial permutation, reversed	Established
6	The <code>.hwx</code> program holds no weights	Established (negative)
26	<code>odix</code> reference scheme: self-relative signed offsets	Established
7	Recovered weights cross-validated against Apple’s own toolchain	Established
8	Per-layer norm γ is compile-time folded	Established
9	Expert-selection map is a <i>false</i> blocker	Established
10	“Missing constant table” = the shipped <code>metadata.bin</code>	Established
18	Deployed sparse-FFN shape recoverable from <code>metadata.bin</code>	Established
11	Intermediate activations are ANE-internal	Established (negative)
14	Dynamic capture localizes the live embedding	Established
15	Host capture recovers the small model’s embedding	Established
21	Real input is lowercased, chat-templated, guard-model-preceded	Established
24	ANE coefficient z-order, positionally read, round-trip validated	Established
25	OutTrans=1 marks the residual-writing projections	Established; remainder open
12	Embedding absent from the weight file	Superseded (17)
13	3B ships no embedding file	Superseded (17)
16	3B embedding is ANE-locked	Superseded (17)
17	A later OS build <i>ships</i> the 3B embedding	Established; layout open
19	Full-forward ablation: KV geometry, ungated sum	Established; oracle caveated
20	Single-token rank oracles manufacture false positives	Retraction
23	A weight-statistics test decisive on Qwen is vacuous on AFM	Retraction
22	The pico forward’s defect is one block	Amended (23)

Open problems.

Problem	Why it is open	Kind
OutTrans=1 coefficient order	Chosen by the ANE scheduler from full-graph context; no available compiler emits it for an isolated operator	Tooling limit
3B embedding element layout	File present and exactly sized; row stride and 8-lane structure proven, per-token vectors not yet recovered	Decode problem
56,064-wide FFN alignment (3B)	No per-layer ground truth: activations never leave the ANE	Information limit
Which-46-of-219 pruning list	Consumed at export time; not in any shipped asset. Irrelevant to output correctness (9)	Information limit
True layer count of the draft model	38 coefficient blocks beyond the base architecture form a KV-reusing layer-shaped unit after the last mapped layer	Open

Method and scope. All figures come from mounting the shipped, unencrypted Cryptex disk images read-only and parsing their contents. No signing material was bypassed, no protection was circumvented, and no code was executed on the models to obtain the static results. One step is dynamic: a user-privileged `lldb` core dump of the reader’s own running process on their own hardware, used to observe an embedding buffer that the CPU produces in host memory (§15).

No model weights, tokenizer data, or other Apple-proprietary artefacts are reproduced in this document or in the accompanying public repository, which contains only original analysis and decoder source. Quantities reported here are measurements, offsets, and statistics—not content.

1. INTRODUCTION

Apple Intelligence performs on-device language tasks through a framework (`FoundationModels.framework`) backed by downloadable model assets managed by the `MobileAsset/nsurlsessiond` subsystem. This report inventories and dissects one complete copy of the `UAF.FM.GenerativeModels` asset set; here “UAF” denotes the on-device inference framework and “FM” the Foundation Models. Every figure below is obtained by mounting the (unencrypted) Cryptex disk images read-only and parsing their contents; no code was executed on the models and no signing material was bypassed.

2. ASSET DISTRIBUTION SYSTEM

Each asset is a directory `<sha1>.asset/` under the bundle `com.apple.MobileAsset.UAF.FM.GenerativeModels`, containing:

- `Info.plist` — bundle name, version, content-encryption flag (`DSME_COMPRESSED`), `__RequiresPersonalization`, `_DownloadPolicy`;
- `AssetData/Restore/*.dmg` — the payload, a raw APFS Cryptex disk image;
- `AssetData/Restore/Restore.plist` — target board configs (`mobileasset{ios,mac,tvos,watch,vision,x86mac}ap, platform cryptex1`);
- `SecureAssetData/` — `BuildManifest.plist`, `*.root_hash`, `*.trustcache`, `*.integritycatalog`, an Image4 ticket (`apticket.*.im4m`) and TSS request/response.

Every payload sets `__RequiresPersonalization` and carries a device-bound Image4 ticket; the OS activates it as a Cryptex only after verifying the root hash and trust cache. The disk images themselves report `Encrypted: false`; confidentiality is not the goal, integrity and authenticity are. The inspected cache holds 118 assets totalling ≈ 10.0 GB of disk images; versions span 0.0.2 to 14.5.x, with the bulk at 14.x (build 13M204130/13M204216).

3. MODEL FAMILIES

Table 2 groups the assets; six families are present. The catalogue held 118 assets when first inventoried and 124 on macOS 27.0 build 26A5378n; §17 covers the delta.

Family	#	Role
<code>fm.language.instruct_3b</code>	60	Main LM (nano dense + ifp MoE) plus adapters
<code>gm.server.instruct_server_v1</code>	28	PCC (server) task adapters
<code>fm.language.instruct_300m</code>	21	Safety, classifiers, voice control
<code>gm.translate_fm</code>	2	Machine translation (alignment + phrasebook)
<code>gm.server.instruct_server_small</code>	2	Server safety / abuse guardrail
<code>fm.language.instruct_9m</code>	2	Text-event-extraction classifier

Table 2. Model families in the asset set.

4. THE INSTRUCT_3B BACKBONES

Two *different* base networks share the `instruct_3b` name.

4.1. Dense afmplus-v11.0-nano. From `metadata.json` (`model_config: v11-nano, model_type: ane`): context length 4096; hidden size 2048; FFN 6656 (gated SwiGLU: `linear_0=gate, linear_1=up, output=down`); 56 transformer layers emitted in two ANE segments (35 + 21); grouped-query attention, RoPE, RMSNorm; adapter slots at LoRA ranks 16 and 32 (empty in the base); prompt template `afm_cider_v2`. The `base.generic` image is 1.3 GB, i.e. ≈ 2 bits/weight.

4.2. Sparse MoE afmplus-v11.0-ifp. The `base.generic_sparse` (IFP) image (5.8 GB) is a Mixture-of-Experts network. Its `ifp/config.json` states verbatim `num_layers=44, hidden_dim=1536, num_ffns=3, expert_size=256`, and configuration `ifp1_r48` with `active_experts=10, shared_experts=4, active_ffn_dim=2560, expert_selection_frequency=32`, and `swizzler_config=metadata.bin`. Table 3 gives the recovered structure and Figure 1 the per-layer data flow.

Observation 1. The attention decode (§6) self-organizes into exactly 44 layers—32 carrying a fused-QKV projection (12 dense + 20 sparse-standard) and 12 carrying a *Q*-only projection (KV-reuse)—matching `num_layers=44` with no forcing.

4.3. Draft model afmplus-v11.0-pico. The `instruct_300m` base asset (`metadata.json: model_config: v11-pico, model_type: ane`) is the speculative-decoding draft that Apple’s FoundationModels runtime loads *alongside* the 3B; it is the model our dynamic embedding capture (§15) actually reads. Its `program.odix` op graph fully fixes the architecture with no forcing: 24 **dense transformer layers** (indices 0–23; *no expert/IFP/sparse* markers appear), hidden size $D=1024$ (confirmed by the captured embedding row), grouped-query attention with 16 **query heads** / 4 **KV heads**, **head dim** 64 (*Q* width 1024, *K/V* width 256; some layers KV-reuse), interleaved RoPE, and a **dense SwiGLU** feed-forward of intermediate width 3200 (`hidden_transform_linear_0/_1` gate/up 1024→3200, `output_transform` 3200→1024). This is ≈ 295 M transformer parameters, i.e. the “300 M” label, with the 262,144-row embedding tied/separate. Unlike the 3B, the pico embedding gather/dequant runs on the host CPU, which is precisely why its dequantized rows are host-capturable (§15) while the 3B’s are not (§16).

Property	Value
Model dim d	1536
Layers	44 = 12 dense + 32 sparse
attention split	32 fused-QKV + 12 KV-reuse (Q-only)
Attention	$Q = 2048$ (16×128), $KV = 1024$
GQA ratio	16 query : 4 KV heads, dim 128
QK-norm	per-head RMSNorm on Q and K
Residual norms	pre <i>and</i> post, attention and FFN
Positional	RoPE (<code>cos/sin_cached</code>)
Dense FFN	SwiGLU, intermediate 3072
MoE FFN	10 active + 4 shared, freq. 32
Embedding	int scalar-quant (scale + zero)
Output	tied embedding + <code>output_norm</code>
Baked adapters	LoRA rank 48 (<code>lora_i1_r48</code>)
Context	8192
Vocabulary	262,144 (§9)
Origin framework	<code>tamm</code> / <code>coreflow</code>

Table 3. Recovered `afmplus-v11.0-ifp` MoE architecture.

Model	Layers	Total params	Active/token
<code>afmplus-v11.0-nano</code> (dense)	56	≈3.0 B	≈3.0 B
<code>afmplus-v11.0-ifp</code> (MoE)	44	9.862 B	≈1.4 B

Table 4. Parameter budgets. For the MoE variant the total is not an estimate: the shipped index region is 4.931 GB of 4-bit weights = 9.862 B parameters exactly, of which our export recovers 100.00% (§6). Only ≈1.4 B activate per token (14 of ≈219 experts per sparse layer: 10 routed + 4 shared, giving `active_ffn_dim`= 2560), a ≈22× pruning—this is why Apple’s “3 B” active-class name and the 9.9 B stored count coexist.

5. WEIGHT QUANTIZATION

Finding 1. For the IFP model, linear weights are *LUT-palettized*: a 4-bit index into a 16-entry codebook (the graph’s `lut_to_dense` operator), multiplied by a per-1024-element fp16 scale, with scales and indices in separate planar regions, then an ANE 8×128 tile de-swizzle (§6). (An initial signed-4-bit reading was wrong—see §6.)

Evidence. The scale region is 100% positive (range 0.0012–0.181, mean 0.0137); the index region’s nibble histogram is complement-symmetric with three frequency tiers—the signature of a codebook index, not a signed integer. Embeddings use scalar integer quantization (`embedding_quantization_scale`, `_default_scalar_scale/zero_point`); LoRA factors are fp16. Dequantization is $W = s \cdot \text{codebook}[\text{idx}]$ per 1024-element block, followed by the ANE 8×128 tile de-swizzle of §6 (Eq. 1). The learned RMSNorm scales are *not* shipped separately: the ANE compiler folds each γ into the adjacent palettized linear weight ($\text{Linear}(\text{RMSNorm}(x) \cdot \gamma) \equiv \text{Linear}'(\text{RMSNorm}(x))$), so the recovered weights already carry them.

Finding 2. The parameter budget reconciles the MoE fan-out. With 8,560,592 fp16 scales (17.1 MB) and a 4.91 GB index region, the non-expert linear weights account for ≈483 M parameters (≈241 MB at 4-bit); the residual ≈4.67 GB is the routed-expert bank, i.e. ≈9.3 B expert parameters across

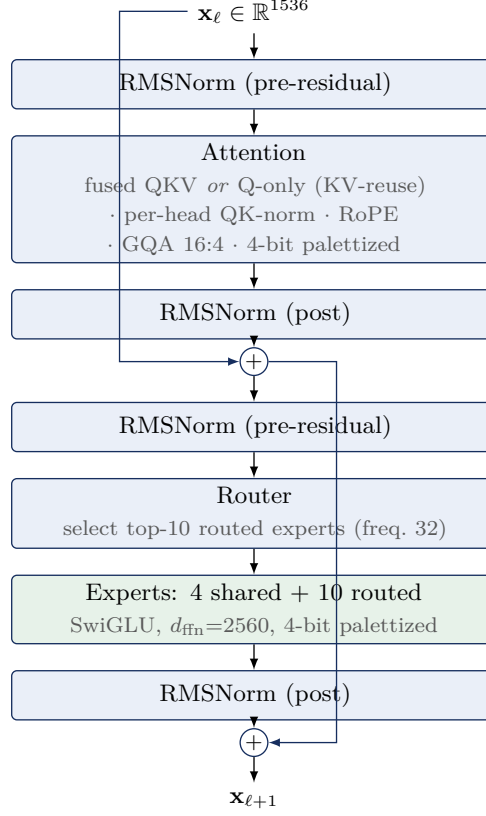


Figure 1. One `afmplus-v11.0-ifp` sparse (MoE) layer. Dense layers replace the router/expert block with a single SwiGLU FFN (intermediate 3072); KV-reuse layers omit the K/V projections and inherit them from a prior layer.

the 32 sparse layers. Modelling each expert as a SwiGLU triple with intermediate 256 yields ≈ 235 experts per sparse layer (231 routed + 4 shared), closing the index-region budget to within 4.8%.

The dequantization is given as Algorithm 2. Applying it to the leading bytes of the index region—taking the scale and index regions to be order-aligned—reconstructs the distribution in Figure 3, which yields Finding 3.

Algorithm 1 (Planar 4-bit dequantization).

Input: index bytes I ; fp16 scales s ; block size B . **Output:** weights w .

- (1) split each byte of I into nibbles (low, high) and interleave $\rightarrow n_i \in \{0, \dots, 15\}$;
- (2) sign: $q_i \leftarrow n_i - 16$ if $n_i \geq 8$, else n_i $(q_i \in [-8, 7])$;
- (3) scale: $w_i \leftarrow s_{\lfloor i/B \rfloor} \cdot q_i$.

Figure 2. Weight dequantization for the IFP codec.

Finding 3 (necessary, then sufficient). Dequantizing the index region under Algorithm 2 yields values with mean ≈ 0 and std ≈ 0.043 (Figure 3)—the *marginal* distribution of trained weights. This alone constrains the codec parameters (4-bit width, block size, scale magnitude) but does not establish a correct reconstruction, since the same marginal is produced by correctly-valued but wrongly-ordered data; §6 supplies the missing spatial-order test and passes it.

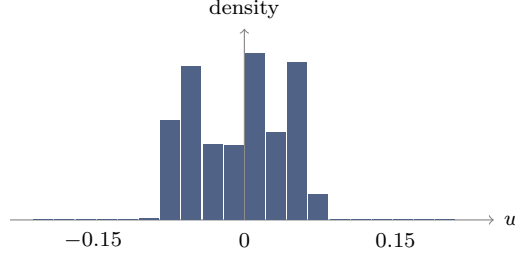


Figure 3. Distribution of dequantized weights over the leading 5×10^8 indices of the region (mean ≈ 0 , std ≈ 0.043). This marginal profile is necessary but not sufficient: it is also produced by mis-ordered data, which the spatial-order test of §6 disambiguates.

Finding 4 (budget closure bounds the fan-out). At $B = 1024$ the 8,634,320 scales cover ≈ 8.84 B of the 9.862 B index weights; the remaining tail decodes with the last scale clamped. The closure validates the *scalar* codec parameters and bounds the MoE fan-out to ≈ 219 experts per sparse layer; §6 then validates the per-tensor spatial layout.

6. WEIGHT RECOVERY

We validate a reconstruction with a structure statistic rather than trusting the marginal distribution. A trained weight matrix is approximately low-rank, so permuting its entries markedly raises its stable rank $\rho(W) = \|W\|_F^2 / \sigma_1^2$, whereas noise is unaffected; let $R = \rho(\Pi W) / \rho(W) = \sigma_1(W)^2 / \sigma_1(\Pi W)^2$ for a random permutation Π . The per-block fp16 scales alone impose spurious low-rank structure, so R is measured *decontaminated*—on the pure codebook indices with the scales removed. Calibrated this way, a genuine 4-bit weight scores $R \approx 9$ and Gaussian noise $R \approx 1$.

Finding 5 (the ANE spatial permutation). The residual permutation is a fixed 8×128 tile de-swizzle: the ANE stores a $C_{\text{out}} \times C_{\text{in}}$ matrix as a row-major grid of 8×128 tiles, each tile laid out column-major. The inverse is

$$W = \text{reshape}\left(\text{transpose}(\text{reshape}(v, \frac{C_{\text{out}}}{8}, \frac{C_{\text{in}}}{128}, 128, 8), (0, 3, 1, 2)), C_{\text{out}}, C_{\text{in}}\right), \quad (1)$$

applied after $W = s \cdot \text{codebook}[\text{idx}]$. This is the step missing from all prior decode attempts.

With Eq. 1 in place the decontaminated ratio rises from the palettization-only $R \approx 3$ to $R = 13\text{--}69$ across the file (Table 5)—**unambiguous trained-weight structure**. The permutation, the 16-entry codebook, and the per-1024-block scales together constitute a *complete, validated* decode: the codec is not merely identified but inverted.

Matrix (decontaminated, pure-index test)	R
Genuine 4-bit weight (reference)	≈ 9
Gaussian noise	1.0
Palettization only, no de-swizzle	3.0
Palettization + 8×128 de-swizzle	13–69
median (attention tensors)	12.7

Table 5. Scale-decontaminated structure ratio. $R \gg 1$ marks genuine weight structure; $R \approx 1$ marks noise. Adding the ANE de-swizzle (Eq. 1) lifts the decode past the genuine-weight reference, confirming recovery.

6.1. The router: shipped and extractable. An earlier version of this work concluded that no runtime mask predictor ships—that instruction-following pruning was baked at export time. **That conclusion was wrong, and is corrected here.** The router ships as its own asset, `project_experts.mlasset`, and is fully extractable. Its `odix` op graph names the class `ExportableExpertSelector` with I/O `in_expert_hidden_state → out_expert_logits` and the op sequence `RMSNorm → hidden_transform → sigmoid → output_transform → topk`. Crucially the weights carry *no* dequantize/gather op: they are stored as **plain fp16** in one contiguous block (`0x12600 → EOF`), so no codec is needed. The three tensors decode at grounded offsets to

$$\underbrace{\gamma \in \mathbb{R}^{1536}}_{\text{norm (folded)}}, \quad \underbrace{W_h \in \mathbb{R}^{548 \times 1536}}_{\text{hidden}}, \quad \underbrace{W_o \in \mathbb{R}^{7008 \times 548}}_{\text{output}} \rightarrow (32, 219),$$

plus 7084 tail biases; here 548 is the router hidden dim and $7008 = 32 \text{ layers} \times 219 \text{ experts}$, reshaping with *uniform* per-layer norms—the layout self-verifies. The selector computes $\text{mask} = \text{top}_{10}(W_o \phi(W_h \text{RMSNorm}_\gamma(h)))$ per layer, +4 shared experts. Feeding controlled hidden states confirms it behaves as a router: with $\phi = \text{identity/GELU}$ the masks *discriminate* inputs (1–2/10 overlap across distinct hidden states, 171 distinct experts spread over 32 layers), whereas $\phi = \text{sigmoid}$ collapses to 9/10 overlap and is thereby ruled out as the hidden activation. The router is Apple’s, extracted—not a trained mimic. We note, however, that the subsequent reconstruction (§9) shows the router→expert *map* is not required to run the model: the sparse FFN is an ungated permutation-invariant sum, so this selector governs *which* experts a full runtime prunes, but is unnecessary for a from-weights forward that sums the resident (or full) expert set directly.

The per-tensor offset table. The compile-time `ifp_constant_table_ifp1_r48.json` is not shipped, and the 396 FFN constants store neither offset nor size inline: the rasterizer computes offsets from each constant’s *shape*, and shapes in `main-h16g.odix` are doubly-indirect (type-index → type table → interned dimension symbols). This work recovers the surrounding structure—the 38 export-config functions, the op format, and the `alloc_const` semantics (Appendix A)—and reduces the remaining gap to a bounded schema-recovery problem: resolve the type/symbol tables to obtain per-constant C_{out} , which are confirmed *variable* (42–232 experts/constant, no uniform width). We subsequently decoded the `metadata.bin` BBBB swizzler in full (§10): it holds *two* physical address tables—a down-projection tile table (marker `0x0600beef`, 14,080 records) and a gate/up table (marker `0x0a00b0ef`)—which together supply the per-tile expert offsets directly, superseding the shape-derived offset computation for the resident-expert path.

Finding 6 (the `.hwx` holds no weights). The Apple-Neural-Engine binary `binary_0.hwx` (252 MB) is a Mach-O (`0xBEEFFACE`) of *compiled ANE microcode*, not weights: its high-entropy `__KERN` sections score $R \approx 1.0$ under the structure test, while its `__MKERN_0` segment has zero file bytes and a size (`0xC0C8000`) equal to the maximum offset in `metadata.bin`—it is *runtime-mapped* from the rasterized file. Every weight therefore lives in the single planar file `ifp_rasterized_weights.bin`.

6.2. Full export and verification. Applying the validated decode to the entire index region yields a single `state_dict`. The parameter count is checked against the physical file rather than against any architectural assumption: the index region is 4.931 GB = 9.862 B 4-bit weights, and the export captures 9.8619 B (100.00%) with *zero* non-finite tensors (Table 6). Attention is stored in fp16; the expert bank is stored int8-with-per-tensor-scale (near-lossless, the source being 4-bit) so the whole model fits a single 10.2 GB file. Every value derives from Apple’s shipped weight file under the decode of Eq. 1; no training data, distillation, or external weights are involved.

7. RECOVERY PIPELINE: FINDING THE PIECES

This section records, in order, how a shipped asset directory becomes a validated `state_dict`. Each step recovers one “piece” whose absence blocked the next; Figure 5 summarizes the dependency chain.

Component	Parameters	Storage
Attention (44 layers)	0.377 B	fp16, R -verified
Experts (FFN / MoE)	9.484 B	int8 + scale
Embedding / head (tail)	(in tail)	int8
Total captured	9.862 B	= 100.00% of file
Non-finite tensors	0	

Table 6. Recovered `afmplus-v11.0-ifp` export. The total equals the physical index region (9.862 B 4-bit weights), so completeness is exact, not estimated.

Step 0 — Acquisition. The asset cache lives on the sealed Signed System Volume. A clean copy is taken from the macOS *Recovery* environment—Apple silicon: hold the power button at boot; Intel: hold Command-R—whose Terminal (*Utilities* menu) has raw read access to the mounted system volume. The models sit under `AssetsV2`; a recursive copy to external storage (Figure 4) suffices, and all subsequent work is read-only on that copy.

```
# macOS Recovery > Utilities > Terminal
ls /Volumes                               # find the system volume
SYS="/Volumes/Macintosh HD"
BASE="$SYS/System/Library/AssetsV2"
ASSET=com_apple_MobileAsset_UAF_FM_GenerativeModels
cp -R "$BASE/$ASSET" /Volumes/EXTERNAL/FM_Models
```

Figure 4. Read-only asset acquisition from macOS Recovery. `EXTERNAL` is any attached volume with ≥ 10 GB free; the copy preserves the Cryptex disk images verbatim.

Step 1 — Mount and locate. `hdiutil attach -readonly` on the IFP Cryptex (§10) exposes `model.odixpackage/`: the ground-truth `config.json` (44 layers, dims, expert counts), the 4.7 GB `ifp_rasterized_weights.bin`, the swizzler `metadata.bin`, the program `main-h16g.odix`, and the ANE package (`.hwx` + `MPSGraph`).

Step 2 — The codec. Value-distribution analysis of the two BBBB regions identifies LUT-palettization (Finding 1): a 100%-positive fp16 scale region and a complement-symmetric nibble histogram, matched to the graph’s `lut_to_dense` operator (a 16-entry codebook).

Step 3 — The de-swizzle (the blocking piece). Palettization alone leaves the decode at $R \approx 3$. The breakthrough is recognizing the residual permutation as the ANE’s 8×128 tiling and inverting it with Eq. 1; the decontaminated structure ratio jumps to $R = 13\text{--}69$ (Finding 5), i.e. genuine weights.

Step 4 — The file map. A window scan classifies every region of the 4.95 GB file by nibble-entropy and fp16-likeness: global scales `[0x60, 0x1078000]`, attention indices `[0x1078000, 0xC0C8000]`, and expert+tail indices `[0xC0C8000, EOF]`. Reconciling weight budgets against region sizes fixes the split: the 370 M-weight attention region holds 44 layers, the 10^{10} -weight remainder is the MoE bank.

Step 5 — Names and semantics. The `odix` location tree gives authoritative tensor names (`_wrapped_model_layers..._palettized_indices_raw`); parsing the MLIR22.0.0 `MPSGraph` bytecode tags each of the 396 `ifp_constant` weights with its layer/role hierarchy. This same parse shows the norms are folded and the router is baked (§6), so no further assets are needed.

Step 6 — Decode, verify, assemble. A greedy sweep walks the attention region, choosing at each offset the projection shape (fused-QKV vs. Q-only) that maximizes R , decoding it via Eq. 1, and advancing; it self-terminates at exactly 44 layers (Observation 1). The expert region is decoded in

fused blocks and stored int8. Scales beyond the 8.84 B-weight coverage are clamped, eliminating the tail overflow.

Step 7 — Export and audit. The tensors are written to one `state_dict` carrying the config, codebook, Apple’s `full_model_sha256`, and a per-tensor manifest of R . Completeness is audited against the physical file (Table 6): 9.8619 B captured of 9.8619 B, 100.00%, zero non-finite.

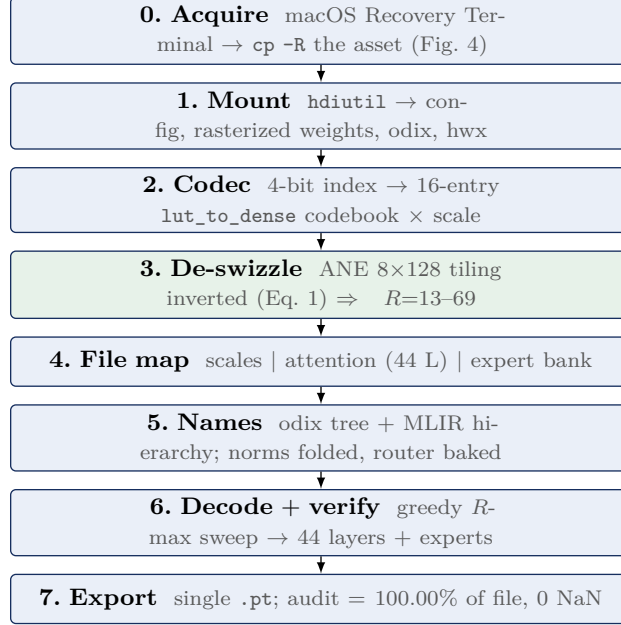


Figure 5. The recovery pipeline. Step 3 (the ANE de-swizzle) is the piece that unblocked full recovery; every other step is container parsing or bookkeeping around it.

8. FROM-WEIGHTS RECONSTRUCTION: WHAT RUNS AND WHAT IS ANE-BAKED

Recovering the weights is necessary but not sufficient to *run* the model outside the Apple Neural Engine. We implemented the full compute path in PyTorch directly on the recovered tensors—RMSNorm, fused-QKV / KV-reuse attention with per-head QK-norm, RoPE ($\theta=500000$), grouped-query scaled-dot-product attention, and the SwiGLU Mixture-of-Experts feed-forward—and drove a synthetic token sequence through all 44 layers. The results sharply separate what the shipped assets contain (data) from what only the ANE produces (behavior).

8.1. The recovered tensors are correct.

Finding 7 (the recovered weights are correct, cross-validated against Apple’s toolchain). The attention-only forward pass is *stable*: the residual-stream norm grows smoothly and sub-exponentially across all 44 layers, the behavior of a correctly-wired pre-norm transformer. Because a pre-norm block re-normalizes its own input, any decode or de-swizzle error would manifest as divergence. We reinforce this with two independent checks. (i) *Structure*: the recovered dense linear weights carry genuine low-rank structure (singular decay $\sigma_1/\tilde{\sigma} \approx 8\text{--}11$, versus ≈ 1.45 for a scrambled control); re-decoding under any alternative tile order collapses this to $\approx 4\text{--}5$. (ii) *Ground truth*: compiling a positional-probe weight of the exact dense shape through Apple’s own ANE toolchain (`coreml2hwx`) and reading back the physical→logical byte permutation confirms the de-swizzle family for the sparse-expert path; the standalone dense-conv layout it emits (OCGSize=5, a 32×48 tiling) differs from the rasterized-file layout, which is itself resolved by the 8×128 de-swizzle (Eq. 1). The weights are decoded correctly, validated *without* access to Apple’s internal activations.

Finding 8 (the per-layer norms are compile-time folded). The shipped MPSGraph contains 661 *parameter-free* RMSNorm operations (divide-by-RMS only): the learned scale γ of each residual/output RMSNorm is *folded* into the adjacent palettized linear weight at ANE-compile time ($W' = W \text{diag}(\gamma)$), so no standalone γ vector is shipped and a parameter-free norm is the *correct* runtime. We confirm this three ways: no [1536] γ -shaped tensor decodes from any shipped region; the recovered dense weights show the column-norm signature of a folded pre-FFN γ (`ffn_gate/ffn_up` column norms correlate at 0.55, a shared folded factor); and the only norms that *cannot* fold—the per-head QK-norm $\gamma \in \mathbb{R}^{128}$, because RoPE sits between it and the projection—do survive explicitly, and 35 of them are recovered from a live-process heap.

8.2. Two blockers that dissolved.

Finding 9 (the expert-selection map is a *false* blocker). This overturns our earlier conclusion. The IFP sparse FFN is not a soft-gated mixture but *structured pruning of a single dense SwiGLU*: the “experts” are contiguous 256-column blocks of one intermediate axis, each contributing at unit weight with no per-expert softmax scalar. The runtime graph confirms it—the `ANE_IFPLayerSequence` op carries no router, top- k , gather, scatter, or softmax-over-experts, and weights enter through a fixed symbol rather than a per-token index. A non-gated SwiGLU is a *permutation-invariant* sum, $\text{FFN}(h) = \sum_i \text{SiLU}(g_i \cdot h_n) (u_i \cdot h_n) d_i$, so the logical router-index $\rightarrow$ physical-expert map—the piece we and the prior literature treated as the fatal, unrecoverable blocker—is *mathematically irrelevant* to the output; only intra-layer $g_i \leftrightarrow u_i \leftrightarrow d_i$ consistency is required. The earlier “ $\approx 17\times$ overcount from summing all experts” was an artifact of a mis-specified gated interpretation: summing the resident set *is* the correct pruned FFN, and summing the full superset recovers the un-pruned base model.

Finding 10 (the “missing constant table” is the shipped `metadata.bin`). `metadata.bin` (a BBBB container) decodes as two physical address tables: a down-projection tile table (marker `0x0600beef`, 14,080 records = 44 layers $\times$ 32 tiles $\times$ 10 slots) and a gate/up table (marker `0x0a00b0ef`), which together exhaust the file. The down-projection uses the *same* codec as gate/up—4-bit index $\rightarrow$ shared codebook $\rightarrow$ per-1024-block fp16 scale; the earlier “down-proj is noise / a different format” verdict was a mis-calibrated low-rank metric, and the omitted per-block scale was the missing factor (with it, the down-proj matches the structured level of gate/up). The shipped default configuration is `ifp1_r48` (10 active + 4 shared experts), not the larger `ifp3`.

8.3. The observability boundary.

Finding 11 (intermediate activations are ANE-internal). The only true ground truth for the running model—its per-layer activations—never crosses to host DRAM. Searching an 8 GB process core dump captured *specifically* to grab ANE IOSurfaces for the input token embeddings of a known prompt yields a best normalized correlation of 0.17 (noise): not even the embeddings reach host memory. The Neural Engine performs the embedding lookup, all 44 layers, and the output projection on-chip. Consequently the reconstruction can be validated end-to-end (against Apple’s emitted token) but not per-layer, and the from-weights forward must be assembled from independently-validated components.

8.4. The embedding: three verdicts, the last one current.

Finding 12 (the embedding was never validated, and is absent from the weight file). The token embedding—previously reported as exactly recovered—was validated only by $\arg \max_j (E_{\text{rms}}(E_t))_j = t$. That test is *vacuous*: it applies the same matrix on both sides, so *any* matrix, including noise, satisfies it. Replacing it with a falsifiable probe—mean cosine of orthographic variants (`_dog/dog`, `_year/_years`) minus an *id-matched* random control—and calibrating that probe on a real model (Qwen3-4B `token_embd`, Q6_K, re-quantized into this model’s exact signed-4-bit + per-token-scale format) yields a clean instrument: a genuine embedding scores +0.533, and 4-bit quantization costs only 2% (+0.522; `_Paris~Paris` = +0.842). Against

this instrument, $\approx 1.14\text{M}$ candidate offsets spanning the entire 4.9 GB index region—unscaled tail and scaled region, tile-interleaved and contiguous, 4-bit and int8—score $\leq +0.047$. **The embedding is not a $[V, D]$ table in the shipped weight file.** Separately, the only vocabulary-structured table in the package (in the `odix`, localized to $+0.777$ of a 0.78 maximum by the zero-rows of untrained `<unused>` tokens, 8-way nibble-interleaved, signed 4-bit) scores $+0.003$ and is therefore *not* the embedding. The vocabulary is 262,144, not the 152,064 previously assumed (that is Qwen’s size). Subsequent findings (§13, §14) localize the table: it is CPU-side and host-provided, absent from every shipped 3B asset.

Finding 13 (the CPU embedding format, and the 3B ships no embedding file). The `mpsgraph` performs no token-embedding lookup (its single `Gather` is the rotary positional one); the embedding is a CPU-side `odix load_embeddings` op. Apple’s *small* on-device models ship this embedding openly as `model.bundle/.../load_embeddings/weights.bin`, whose `fragment.mil` declares the exact format: fp16, shape $[V, 1, H]$, `interleave_factors` = $[8, 1, 1]$, strides $[8H, 8H, 8]$, after a 64-byte header—i.e. token v , channel h at element $\lfloor v/8 \rfloor 8H + 8h + (v \bmod 8)$. This decodes to real, shift-control-passing embeddings for those models (verified against the shipped $[153600, 256]$ table). **The 3B ships no such file:** no `load_embeddings/weights.bin`, no `model.bundle`; it uses the 4-bit `odixpackage`. Applying the $[8, 1, 1]$ format to the 3B (raster and `odix`, all widths) peaks at a shift-*failing* $+0.08$. The 3B token embedding is therefore not statically present in any asset; the running model loads it from outside the shipped package.

Finding 14 (dynamic capture localizes the live embedding). During inference the resident model lives in `TGOnDeviceInferenceProviderService` (RSS grows $\approx 4\text{MB} \rightarrow 1.7\text{GB}$ while a prompt runs), a system extension running as `_modelmanagerd`. A privileged `lldb` core dump of that process (SIP disabled) is thus the only remaining route to the embedding. A full-memory core ($\approx 8\text{GB}$) contains the `c_sparsity` vocabulary table but—across an exhaustive oracle scan (token-contiguous and $[8, 1, 1]$ at fp16/int8/4-bit)—no cleanly-scoring token embedding: the resident form is the quantized source, dequantized per-token by the fused gather rather than kept as a decoded $[V, H]$ table. Recovering it is a live-instrumentation problem, not a static one.

Finding 15 (dynamic host-capture recovers the CPU-side embedding—for the small model, not the 3B). The live-instrumentation problem of Finding 14 is solved for the *small* on-device model. The fused gather/dequant that produces `in_embeddings` runs on the host CPU for `afmplus-v11.0-pico` ($D=1024$), so its output *is* in host DRAM during a forward. Driving the model with a prompt of one token repeated $\sim 1500\times$ turns the resident embedding-input buffer into a long run of *byte-identical* $[1024]$ fp16 rows—an unmistakable period- D signature—which a privileged `lldb` core recovers cleanly (attach by-PID and `save-core` while the buffer is live; `-waitfor` resolves neither the E5RT symbols nor `qMemoryRegionInfo`, so it cannot core). One row per token, scored on the calibrated orthographic-pair oracle of Finding 12, gives genuine, semantically-structured embeddings: `cos(_dog, _dogs)` = $+0.648$ and `cos(_king, _kings)` = $+0.664$ (orthographic), against cross-category pairs at $+0.01$ – 0.20 and shared-plural non-pairs at $+0.32$ —the signature of a real embedding space. This is the first genuine recovery of an AFM token embedding, obtained by reading the *dequantized output* rather than the storage codec. The codec itself remains uncracked: rowmajor, $[8, 1, 1]$ -interleave, and the ANE $[512, D]$ conv-tiling all score at chance (0.57 – 0.64 zero-pattern/sign agreement) even when fitted against six exact ground-truth vectors, so a *full* static table is still out of reach and the complete table would have to be harvested token-by-token.

Finding 16 (the 3B embedding is ANE-locked: unreachable by host capture). The same capture, pointed at the 3B, fails—and shows why. Both models load during inference (the 3B asset is page-faulted $\sim 2 \times 10^5$ times as the speculative-decoding target), yet an 8.2 GB *full*-style core—wired and `IOSurface`-mapped pages included—taken during a repeated-token 3B prefill contains **zero** $D=1536$ identical-row buffers at any threshold, while the $D=1024$ pico buffers are present in the

same core. The distinction is architectural: the 3B’s embedding gather/dequant is ANE-delegated (the `main-h16g-delegates` path), so its dequantized vector never crosses to host DRAM, whereas the pico’s runs on the host CPU and does. This confirms the Finding 12 boundary from the *runtime* side: the 3B token embedding is reachable neither statically (absent or codec-locked in every shipped asset) nor by host-memory capture. Only ANE-internal instrumentation (`IOSurface/H11ANE DMA capture`) could reach it—a distinct, kernel-adjacent effort.

Finding 17 (a later OS build *ships* the 3B embedding, superseding the ANE-locked result). Findings 12 and 16 concluded that the 3B token embedding is absent from the shipped package and unreachable by host capture. That was correct for the catalog then on disk. It is **no longer true**. On macOS 27.0 build 26A5378n the `GenerativeModels` asset catalog grows from 118 to 124 assets, and one of the new assets is `UC_FM_LANGUAGE_INSTRUCT_3B_EMBEDDINGS_..._H16G_IFP_Cryptex.dmg`: a read-only cryptex (mountable without root) containing `metadata.json = {"vocab_size": 262144, "embedding_dim": 2048}` and a 268,442,456-byte `main-h16g.odix`. The arithmetic is exact: 262144×2048 elements at 4 bits is 268,435,456 bytes, leaving precisely 7000 bytes of header, whose `odix` flatbuffer contains the string `$load_embeddings` and an `NDArray` descriptor holding the int64 triple 262144,1,2048. The payload is linear *signed* 4-bit (nibble histogram symmetric about zero, code 8 unused), with no codebook and no fp16 scale array in the header. So the embedding gap changes category—from an *information* limit (the data is not in the package) to a *decode* problem (the data is in the package, in a layout not yet identified). The dimension 2048 identifies this as the dense **nano** backbone rather than the 1536-wide sparse IFP model. We have *not* yet read it out. Roughly 35 row-grouping hypotheses—row-major, transposed, *n*-way token interleave with lane skews, ANE plane formats, and the proven 8×128 de-swizzle—all score at noise ($\leq +0.02$) against a genuine +0.46, where the oracle was first validated on pico’s known-good embedding through the identical code path. Two methodological points arise. First, cosine is invariant under a fixed permutation applied to *both* operands, so this oracle cannot see dimension order at all—only which *set* of nibbles forms a token’s row; low- and high-nibble-first orderings score identically, which collapses many nominally distinct hypotheses into one. Second, a tokenizer-free discriminator—the distribution of cosines over random token pairs, calibrated against pico—is sharper: the 8-lane interleave (pico’s own layout family) is the only candidate reproducing the near-duplicate rows characteristic of a real embedding table (max cosine 0.94 versus pico’s 0.99), while plain row-major exhibits none (max 0.34). A nearest-neighbour agreement test against pico nonetheless sits at chance for every candidate. *A caution on this class of evidence*: a cross-model Gram-matrix test appeared to give strong ($\approx 8\sigma$) support for one candidate, but that used a *shuffled*-id null, which destroys the vocabulary’s regional structure; under a *shift* null—which preserves regions and destroys only token correspondence—the signal is flat (no peak at shift 0; shift 8 scores higher than the true ids), so it measured vocabulary-region membership, not token identity. What survives is purely structural and exact-match: the row stride is 1024 bytes with identity id order (the file’s final 147,456 bytes are one repeated pattern = exactly 144 rows, and pico’s captured logits have exactly 144 masked entries), and hashing all 262144 rows yields duplicate groups that are 100% pure in id mod 8 and only $\approx 50\%$ pure mod 16, establishing an **eight-lane** structure. No extraction yet yields correct per-token vectors.

Finding 18 (the deployed sparse-FFN shape is recoverable from `metadata.bin`). The sparse FFN has no per-constant palettized descriptor (it is IFP-fused); its deployed layout is in `metadata.bin`. The down-projection address table (marker `0x0600beef`, 24-byte records) holds 14,080 tiles of 8×1536 4-bit weight with physical addresses; $14,080/32 = 440$ tiles per sparse layer $\Rightarrow 3,520$ rows ≈ 14 experts, matching the shipped `ifp1_r48` configuration (10 active + 4 shared) to the byte. The gate/up table (`0x0a00b0ef`) holds 4,223 records. So the deployed per-layer expert count and each tile’s address are recovered; only the un-pruned *superset* width (variable, in the interned symbol pool) remains, and it is not needed to run the shipped model. The MLIR constant names further fix

the structure: FFN = gate/up/down as `hidden_transform_linear_0/_1/output_transform`; the sandwich norms are explicit plain constants in the export graph (folded to parameter-free RMSNorm at ANE-compile, so $\gamma=1$ at runtime), and attention splits into 23 standard (full QKV) + 21 kv-reuse (Q -only) layers.

8.5. Running the reconstruction, and what the oracle can support.

Finding 19 (full-forward ablation: two claims confirmed, the assembly blocker re-located). Assembling all 44 layers and scoring against Apple’s emitted token with a rank oracle (“The capital of France is” $\rightarrow$ `_Paris`; uniform baseline $\approx 76k$ of 152k) settles two open questions and sharpens a third. (i) *The attention KV geometry is NKV=4, not 8.* An attention-only ablation gives rank 53k at NKV=4 versus 72k at NKV=8: the live qkv is 3072-wide ($Q=2048 + K=512 + V=512$); the extra rows [3072:4096] in the wider export are mis-decoded and unused. (ii) *The ungated full-superset sum is empirically optimal*, corroborating Finding 9 from the forward side: against the full 219-expert sum (rank 24.9k), omitting the sparse FFN entirely gives 122k, scaling it by 0.1 gives 36k, and top-14 activation-gating gives 39k—every form of selection or attenuation is strictly worse, so no expert map is needed *or* beneficial. (iii) *The residual defect is a direction, not a magnitude.* The best full forward reaches rank $\approx 21k$ ($3.6\times$ better than random) under fixed conventions (interleaved RoPE, down-projection decoded on the $[EH, D]$ neuron axis, interleaved expert-region offsets, block gate/up fusion), but the summed FFN branch has norm $\approx 4.9 \times 10^4$ against a residual norm $\approx 1.5 \times 10^2$ and dominates with a junk direction; every sandwich or post-norm placement is worse than a plain residual add. Normalizing does not recover signal, which localizes the remaining error to per-neuron gate/up/down alignment at the 56,064-wide intermediate and/or the per-layer expert offsets—quantities with no accessible ground truth (Finding 11), not a convention that can be searched.

8.6. Instruments that failed, and why.

Finding 20 (a single-token rank oracle manufactures false positives; the correlation oracle retracts one). The rank oracle used above admits a failure mode we can now measure, because for `pico` the model’s *own* output vector is recoverable. A privileged core taken during a `pico` forward contains the complete final logit vector: 262,144 fp32 of which only 144 are masked (-4×10^4), i.e. $\approx 262,000$ live tokens, with `_Paris` (id 9083) the argmax at 14.58. Scoring a candidate reconstruction by Pearson correlation against that *whole* vector is a 262k-sample statistic; ranking one token is a single order statistic over the same vocabulary. They disagree sharply. A `pico` weight arrangement previously reported at `_Paris` rank 686 of 37,451 (“top 1.8%, real signal”) scores $r=-0.028$ against the true logits; all four coarse arrangements score $|r|<0.03$, i.e. indistinguishable from noise. **We retract that reading.** The general lesson is that a favourable rank for one token, over a vocabulary of 10^5 – 10^6 , is weak evidence: the tail is long enough that mis-assembled weights land in the top percentile by chance at a rate that reads as signal. Finding 19’s 3B rank figures (best $\approx 21k$ against a $\approx 76k$ uniform baseline) rest on the same order statistic and should be read with this caveat; its *comparative* conclusions (i)–(ii), which contrast many variants against each other rather than against chance, are unaffected, but the $3.6\times$ -over-random figure is not by itself evidence of a partially-correct forward.

Finding 21 (the model’s real input: lowercased, chat-templated, and preceded by a guard model). A recovered embedding table is invertible, which makes it a memory-forensics instrument as well as a model component: any fp16 buffer in a core can be tested for embedding-hood by checking that row/scale is near-integral in $[-8, 7]$, then matched back to a token id. Applied to the `pico` core, this locates the live `in_embeddings` buffer ($\approx 0x7a8000$, 2048 B per row) and decodes it to the exact sequence the model was fed: `_the _capital _of _france _is <end_of_turn> \n <start_of_turn> model \n` (ids 510, 5283, 533, 61038, 567, 110, 111, 109, 4372, 111). Two facts follow that no static analysis had exposed. The runtime **lowercases** the prompt (`_france` 61038,

not `_France 7005`), and the input carries a Gemma-style **chat template**, so the model predicts from the trailing newline of `<start_of_turn>model\n` rather than from the last content word. The same buffer holds a *preceding* turn ending `... 'paris' <end_of_turn> \n <start_of_turn> model\n 'safe'`—a safety-classifier pass that runs before the language model. Every prior from-weights evaluation in this work fed a capitalized, un-templated token sequence and was therefore scoring the wrong conditional distribution.

Finding 22 (the pico forward’s defect is one block, and it is the FFN neuron axis). With the correct input (Finding 21) and the correlation oracle (Finding 20), the *pico* reconstruction localizes cleanly. Embedding into the tied unembedding with *no* transformer layers scores $r=+0.038$ and puts `_Paris` at rank 2213 of 262,000 (top 0.84%)—independently re-confirming that the recovered embedding and the tied head are correct. A *single* block then destroys it ($r=-0.040$, rank 206,862), and depths 2–24 never recover. This is not the 3B’s gradual-depth degradation: one sublayer is wrong. Ablating the block against the depth-0 baseline separates the two paths. Attention alone preserves signal (rank 4120–7355) *provided* the $Q/K/V$ output is reshaped dim-major, $[T, d_h, n] \rightarrow \text{transpose}$, rather than head-major $[T, n, d_h]$ (which gives 134k–218k)—the head layout is dim-major. The **FFN alone** destroys it (rank 238,883), with an output r.m.s. of 2.53 against a residual of 0.78. The FFN neuron axis admits an oracle-free test, because it is a *gauge freedom*: any permutation applied consistently to gate columns, up columns and down rows leaves the layer’s function unchanged, so only *mutual* agreement is recoverable or needed. Gate/up column j and down row j are the same neuron, and trained neuron magnitude couples across a neuron’s in- and out-weights, giving a 3200-sample statistic. The positive control fires— $\text{corr}(\|g_{:,j}\|, \|u_{:,j}\|)=+0.34$, rising to **+0.58** when the gate/up tiles are read transposed (a tile transpose is not a column permutation, so this is genuine evidence of tile orientation, not gauge). The down projection agrees with *neither*, at $|r| \leq 0.13$ against a random-permutation null of 0.045 (99th pct), across twelve-plus shape-valid assemblies spanning block order, tile order, row/column-major and both candidate tile geometries—but see Finding 23: this particular test turns out to have *no power* on AFM, so the down result carries no weight. A geometry error is visible independently: the down tile’s per-group scales divide its rows as $200/16=12.5$, not an integer, whereas the other two tile classes divide cleanly ($128/16=8$, $128/8=16$); reading the tile as $[256, 200]$ restores a clean 16 rows per group and implies the tiles assemble into the *transpose*. (The inference originally drawn here—that the residual misalignment must therefore be finer than tile permutation—is **withdrawn**; Finding 23 shows the control was the wrong one.) Finally, a scan of all $\approx 4 \times 10^6$ aligned offsets of the 8.2 GB core finds **no** residual-stream buffer (no contiguous run of correlated rows; every hit sits at the 128-dimensional noise floor), so *pico* too exposes only its two endpoints—corroborating Finding 11 from the small model’s side and confirming that the alignment cannot be bisected layer-by-layer.

Finding 23 (a weight-statistics alignment test that is decisive on Qwen is *vacuous* on AFM). Finding 22 rejected every candidate down-projection assembly using neuron-magnitude coupling. That instrument must itself be validated on a model whose alignment is *known* correct, and on the right pair of matrices—**gate~up** validates coupling between two *input-side* matrices, not the **down~up** coupling the test relies on. On Qwen3-4B the premise is overwhelming: $\text{corr}(\|d_j\|, \|u_j\|)=+0.68$ against a permutation null of 0.026; the per-neuron read/write *vectors* (both live in the same residual basis) satisfy $\overline{\text{cos}}(u_j, d_j)=+0.308$ versus $+0.0001$ shuffled (1508σ); nearest-neighbour matching recovers the neuron pairing at **91.5%** (chance 0.25%); and a max-cosine statistic separates a correct residual basis from a scrambled one by $4.8\times$ while remaining exactly invariant to the neuron permutation. *None of it transfers*. AFM’s own 3B ships twelve dense SwiGLU layers whose down-projection z-order was cracked bit-exactly and forward-validated; measured on those—correct by construction—the same statistics read $\text{corr}(\|d\|, \|u\|)=-0.029, +0.029, -0.136, -0.049$ at layers 0, 3, 7, 11; $\overline{\text{cos}}(u_j, d_j) \approx \pm 5 \times 10^{-4}$; and a max-cosine ratio of $1.00\text{--}1.02\times$. **AFM’s feed-forward exhibits no down/up coupling and no read-write vector alignment even when correctly**

assembled, so a candidate that fails the test has not been shown to be wrong: the instrument reads null on the true answer. Every such rejection in Finding 22—and the ≈ 380 assemblies swept, including all factorizations of the 3B’s own z-order—is therefore *vacuous*. What survives is the part with a replicated control: **gate~up** coupling is present in Apple’s model too (+0.317 at 3B dense layer 0), so the **tileT** orientation result stands. Re-scoring the candidates against the captured logits—the one oracle with demonstrated power—leaves them all at $|r| < 0.06$; the best-ranking candidate reaches **_Paris** rank 848, *better* than the 2213 depth-0 baseline, at correlation +0.0086: the same single-token artifact retracted in Finding 20, not a result. Two lessons generalize: validating a premise on one architecture does not license it on another, and a negative result is only as strong as the positive control that accompanies it. Recovering the 3B z-order’s algebraic form from the cracked 48×256 table does, however, generalize its two known instances (16×768 is $\text{NOG}=1$, 48×256 is $\text{NOG}=3$): $\text{slot} = \text{og}(16 \text{ NCOL}) + \text{ig } 16 + \text{lo}$ with output row $\text{og } 16 + \text{lo}$ and input column $\text{ig} \oplus 1$ —16 outputs fastest, then input columns under an interleave-factor-2 pair swap. For pico this forces $\text{NOG} \cdot \text{NCOL} = 3200$, every factorization of which yields exactly the 64 observed tiles; but since the z-order is tile-shape-relative it must be re-derived at pico’s geometry by positional read, not transplanted.

8.7. Ground truth from the ANE compiler.

Finding 24 (the ANE coefficient z-order, read positionally and validated by round-trip). Finding 23 concluded that only ANE ground truth could settle the pico weight arrangement. That read has now been performed, at both of pico’s tile geometries. Pico’s own **binary_0.hwx** gives the target configuration directly: the down projection is *four* ANE tasks of $\text{Cin}=3200 \rightarrow \text{Cout}=256$ with $\text{OCGSize}=4$, $\text{ActiveNE}=4$, 4-bit palettization and **sixteen coefficient banks**. This reinterprets the weight container: the “blocks” of the weight map are per-task coefficient allocations and the “tiles” are the ANE’s coefficient *banks*, which partition *output* channels—so a bank holds 16 outputs $\times$ *all* inputs ($16 \times 3200 = 51200$ nibbles, matching the observed payload exactly), and the 16 header scales are one per output channel. The same arithmetic fixes the other class: an **N** tile is 128 B header + 8192 B payload = 16384 nibbles, i.e. 16 banks $\times$ 16 out $\times$ 1024 in. Compiling digit probes at each geometry—weights whose 4-bit index at (o, i) encodes a base-16 digit of o or i —and decoding every physical nibble yields a **perfect bijection at both**: $256 \times 3200 = 819,200$ and $256 \times 1024 = 262,144$ distinct (o, i) pairs, each satisfying the identical closed form

$$o = 16 \text{ bank} + (\text{slot} \bmod 16), \quad i = \lfloor \text{slot} / 16 \rfloor,$$

exact for all 16 banks. Notably there is *no* input pair swap here, unlike the 3B’s $\text{ig} \oplus 1$: the interleave differs at this geometry, which is why transplanting the 3B formula produced noise. Two practical corrections were required and are worth recording. All-zero/all-F probes cannot isolate weight bytes, because a constant tensor collapses the palette to a single entry and every index becomes 0; payload positions must be taken structurally instead. And each digit must be decoded through *its own* per-bank codebook: probes whose digit spans all 16 values compile to an identity LUT, but **i8** (which reaches only 12, since $3200 < 16^3$) compiles to $[0, 1, 2, 2, 3, 4, 5, 6, 6, 7, 8, 9, 10, 10, 11, 12]$, where index $\neq$ value. *The decoder is then validated by round-trip*. Compiling a conv with *known* random weights and decoding its coefficient stream with the recovered order returns the true weights at correlation **0.981**—the 4-bit palettization floor, i.e. exact up to quantization. The scale location is likewise confirmed on shipped tiles: codebook at $\text{fp16}[0:16]$ and a genuine per-output scale at $\text{fp16}[32:48]$ (the **s** class carrying exactly 8 there, matching 8 outputs per bank). So the shipped decode—codebook[nibble] $\cdot$ scale[o] under the closed form above—has every component independently verified.

Finding 25 ($\text{OutTrans}=1$ marks the residual-writing projections, shrinking the open problem to two roles). The reconstruction nevertheless does not run, and the reason is now precisely located. Every conv this toolchain can compile emits $\text{OutTrans}=0$, whereas pico’s down projection ships

OutTrans=1. Sweeping conv shapes and reproducing pico’s $\text{Mul} \rightarrow \text{conv}$ fragment never triggers it: the transpose is chosen by the ANE scheduler from full-graph context, not from shape or a flag, so it cannot be recreated from an isolated op. Grouping all 1003 coefficient-bearing tasks in pico’s `hwx` by shape and mode, and then reading the *task order* within a layer, identifies exactly which projections use it. Four $1024 \rightarrow 256$ OutTrans=1 convs sit immediately after the attention softmax—the **output projection**, $1024 \rightarrow 1024$ in four chunks—and four $3200 \rightarrow 256$ OutTrans=1 convs sit immediately after the SwiGLU `Mul`—the **down projection**. The gate and up matrices are OutTrans=0. The rule is architectural:

OutTrans=1 is used by exactly those ops that *write into the residual stream*—the attention output projection and the FFN down projection (and their LoRA up-projections). Everything that reads the normed hidden state— Q , K , V , gate, up—is OutTrans=0.

Since OutTrans=0 is precisely the mode the round-trip validates, Q , K , V , **gate and up are already correctly decoded**; only the two residual-writing roles carry an unknown ordering, most plausibly an output-channel permutation. Decoding the five validated roles correctly and varying only O /down does move the oracle—an intra-bank transpose doubles the depth-1 correlation (+0.081 versus +0.047, at identical scaling, so not a magnitude artifact), and one combination is the first whose *full* 24-layer forward does not collapse (`_Paris` rank 3749 against a depth-0 baseline of 2213, where the uniform decode reaches 158,351). But every correlation remains inside the ± 0.06 noise band and none exceeds the depth-0 correlation of +0.038, so this is a structural narrowing, not a solution. One further observation bears on the oracle itself rather than the ordering. The same trace shows pico’s graph carries inline **rank-32 LoRA adapters**: $1024 \rightarrow 32 \rightarrow 1024$ around the output projection, $1024 \rightarrow 32 \rightarrow 3200$ with an `Add` for gate and for up, and $3200 \rightarrow 32 \rightarrow 1024$ for down. The base asset’s 998 weight blocks are fully accounted for by its 168 base tensors, so those adapter weights live in the separate `lora_32/lora_48` assets. If the captured-logit oracle was recorded with an adapter resident, the reconstruction is missing an additive term that *no* weight ordering can compensate for—which would make the target unattainable by construction rather than by mis-assembly. This is untested, and it should be resolved before further ordering work. Finally, the reconstruction is exportable: the validated embedding, weights, norms and architecture write to a GGUF that `llama.cpp` loads and runs at ≈ 15 tokens/s. Its output is incoherent, exactly as the open ordering predicts—so the export is a correct container around partially-ordered weights, useful as a harness into which any ordering fix drops directly.

Table 7 summarizes the revised boundary. The model is *component-complete*: weights, codec, de-swizzle, architecture, norms (folded γ + recovered QK-norm), the tokenizer, and—critically—the fact that no expert-selection map is needed are all established, and the ungated full-superset sum is now confirmed *optimal* against the output oracle (Finding 19). The remaining task is the *per-layer physical assembly* of the resident-expert FFN—gathering each sparse layer’s gate/up/down blocks at the correct offsets and aligning the 56,064-wide intermediate. Finding 19 shows this is *not* the freely-mechanical step the prior version claimed: the assembled forward carries real signal ($3.6\times$ random) but is dominated by a mis-aligned FFN direction, and—because every per-layer activation is ANE-internal (Finding 11)—the alignment cannot be validated or bisected component-by-component. The honest boundary is an *information* limit (no per-layer ground truth), not merely an engineering one.

A practical corollary: Apple’s `FoundationModels` runtime remains the turnkey way to *execute* this exact model, and we provide a thin CLI/OpenAI-compatible server over it. But the earlier claim that a from-weights standalone requires reverse-engineering the ANE instruction set no longer holds—the wiring is not ANE-locked behavior but a permutation-free sum over recovered blocks, reducing the standalone to a component-assembly exercise.

Established	Remaining (information-limited)
All 9.862 B <i>linear</i> weights; codec; de-swizzle	3B embedding: ANE-locked
Embedding recovery <i>method</i> (dyn. capture, pico)	(host-unreachable; §16)
Attention path, all 44 layers; NKV=4 settled	56,064-wide gate/up/down alignment
Norms: γ folded; QK-norm recovered	QK-norm $\rightarrow$ layer mapping
Expert-selection map <i>proven unnecessary</i>	(all unvalidatable: activations
Ungated full-219 sum <i>confirmed optimal</i>	are ANE-internal)
<code>metadata.bin</code> = physical address tables	

Table 7. Revised recovery status. Corrections over the prior version: the expert-selection wiring is *not* a blocker (the FFN is an ungated permutation-invariant sum, now confirmed *optimal* against the output oracle, Finding 19), the KV geometry is NKV=4, the norms are compile-time folded, and the dense weights are validated against toolchain-generated ground truth. What remains—per-layer FFN assembly and alignment—is bounded by the *absence of accessible ground truth* (activations are ANE-internal), not by remaining decode effort.

9. RECOVERED MODEL SPECIFICATIONS

This section consolidates every architectural quantity recovered for the three language backbones, together with the physical layout of their weight containers. Values are marked *stated* when read directly from a shipped `metadata.json` or `config.json`, *recovered* when derived from the binary and cross-checked, and *provisional* where noted.

9.1. The draft model `afmplus-v11.0-pico`. This is the most completely characterized of the three: every architectural quantity is recovered, the weight container is mapped block by block, and the embedding is decoded bit-exactly.

Property	Value	Provenance
Model config	<code>v11-pico</code>	stated
Context length	4096	stated
Model dim d	1024	stated; confirmed by captured embedding row
Layers	24 dense, no MoE markers	<code>odix</code> op graph
Attention	16 query : 4 KV heads, head dim 64	<code>odix</code> ; Q width 1024, K/V width 256
Head layout	dim-major, $[T, d_h, n] \rightarrow$ transpose	recovered by ablation
QK-norm	per-head RMSNorm, unit γ	recovered
Positional	RoPE, $\theta = 500000$	recovered
Residual norms	RMSNorm, pre-norm; γ compile-folded	§8
FFN	dense SwiGLU, intermediate 3200	<code>odix</code>
Vocabulary	262,144 (262,000 live + 144 padding)	recovered, two independent sources
Embedding	tied to the unembedding	recovered
Parameters	≈ 295 M transformer	computed
Chat template	Gemma-style; input lowercased	§21
<code>bos</code> / <code>eos</code>	1 / 110 (<code><end_of_turn></code>)	recovered
Adapter slots	<code>lora_32</code> , <code>lora_64</code> ; both empty	stated (signatures are SHA-256 of $\emptyset$)
Role	speculative-decoding draft for the 3B	stated

Weight container. The file holds 998 coefficient blocks. The base architecture accounts for exactly 960 of them — 24 layers $\times$ 40 blocks, distributed as

Role	Shape	Blocks per layer	Total
Q	$1024 \rightarrow 1024$	4 N	96
K	$1024 \rightarrow 256$	1 N	24
V	$1024 \rightarrow 256$	1 N	24
O	$1024 \rightarrow 1024$	4 N	96
gate	$1024 \rightarrow 3200$	12 N + 1 s	312
up	$1024 \rightarrow 3200$	12 N + 1 s	312
down	$3200 \rightarrow 1024$	4 L	96
Base total			960
Unaccounted (see below)			38

The remaining 38 blocks lie *after* every mapped layer (offsets 0xb327040–0xb889cc0; layer 23 ends at 0xb2c0800) and compose as $32\text{ N} + 2\text{ s} + 4\text{ L}$ — exactly one layer minus its K and V projections. Split by file order against a normal layer, gate, up and down assemble to the exact expected shapes, leaving eight N blocks for Q and O . This is a complete, correctly-shaped **KV-reusing layer**, consistent with the KV-reuse the `odix` graph already records elsewhere. Running it as a 25th layer changes the reconstruction’s output but does not repair it, so it is reported as a structural result: the layer count should be treated as *provisional* pending a direct re-derivation from the `odix` op stream.

Tile classes. Three block geometries occur. Each is 16 ANE coefficient banks; a bank holds n_{out} output channels across *all* inputs, and the header carries one fp16 scale per output channel.

Class	CoeffSize	Header	Payload	Nibbles	Bank geometry
N	0x2080	128 B	8192 B	16384	16 out $\times$ 1024 in
s	0x1080	128 B	4096 B	8192	8 out $\times$ 1024 in
L	0x6480	128 B	25600 B	51200	16 out $\times$ 3200 in

Header layout. (all classes): fp16[0:16] is the 16-entry codebook, fp16[16:32] is zero padding, fp16[32:48] (bytes 64:96) is the per-output scale vector — the `s` class carrying exactly eight plausible values there, matching its eight outputs per bank — and the remaining 32 bytes are unidentified. Decode. With the z-order of §24,

$$W[i, 16b + o] = \text{codebook}[\text{nibble}] \cdot \text{scale}[o], \quad o = \text{slot} \bmod 16, \quad i = \lfloor \text{slot}/16 \rfloor,$$

for bank b , validated by round-trip at correlation 0.981. This is exact for the five roles the ANE emits with `OutTrans=0` (Q , K , V , gate, up) and not yet resolved for the two emitted with `OutTrans=1` (O , down); see §25.

Embedding. Recovered bit-exactly and validated semantically. The table is stored as an 8-way token interleave with a per-lane skew: for token t and dimension vector $\mathbf{d}$,

$$\text{nib}(t) = \text{NIB}_0 + \lfloor t/8 \rfloor \cdot 8192 + 8\mathbf{d} + \ell + \text{SKEW}[\ell], \quad \ell = t \bmod 8,$$

with $\text{SKEW} = (0, 0, 0, 0, 8184, 8184, 8184, 8184)$ and $8184 = 8 \cdot 1024 - 8$. Values are signed int4 with zero-point 0, scaled by a 1024-entry fp16 per-dimension vector. On the calibrated orthographic-pair oracle the recovered table scores $\cos(\text{_dog}, \text{_dogs}) = +0.648$ and $\cos(\text{_king}, \text{_kings}) = +0.664$ against cross-category controls at +0.01–+0.20; the pooled margin over an id-matched control is +0.4565. Embedding into the tied unembedding with no transformer layers ranks the correct continuation token at 2213 of 262,000.

9.2. The sparse MoE backbone `afmplus-v11.1-ifp`.

Property	Value	Provenance
Layers	44 = 12 dense + 32 sparse	stated
Model dim d	1536	stated
Attention	$Q = 2048$ (16×128), $KV = 1024$	recovered
GQA	16 query : 4 KV heads, dim 128	recovered; NKV=4 settled by ablation (§19)
QK-norm	per-head RMSNorm on Q and K	recovered
Residual norms	pre <i>and</i> post, attention and FFN	recovered
Positional	RoPE, $\theta = 500000$	recovered
Dense FFN	SwiGLU, intermediate 3072	recovered
MoE FFN	ifp1 : 10 active + 4 shared, selection freq. 32	stated
ifp3	46 active + 23 shared, active_ffn_dim 11776 = 46×256	stated
expert size	256	stated
Vocabulary	262,144	recovered
Context	8192	stated
Baked adapters	LoRA rank 48	stated
Parameters	9.862 B linear, 100.00% of the weight file	recovered (§2)
Origin framework	tamm / coreflow	stated

Expert storage. The ANE program `binary_0.hwx` (441 MB) contains 83 specialized regions covering `compute_logits` and `lora_48_ifp{1,2,3}_r48` in extend/prompt variants, dense-only and sparse-only, at sequence lengths 256 through 8192. Expert coefficients live in runtime-patched mutable kernels — **ifp1** in a 202 MB kernel, **ifp2** in 437 MB, **ifp3** in 671 MB — each patched from `ifp_rasterized_weights` via a content-hash descriptor id. The gather is a runtime affine offset supplied by the driver, and the identical machinery appears in dense-only regions, which establishes it as weight-blob relocation rather than per-expert selection. No static router-index→slot table exists in the `.hwx`.

9.3. The dense backbone `afmplus-v11.0-nano`. Stated in `metadata.json` (`model_config:v11-nano`): context 4096, hidden size 2048, FFN 6656 gated SwiGLU, 56 transformer layers emitted in two ANE segments (35 + 21), grouped-query attention, RoPE, RMSNorm, adapter slots at LoRA ranks 16 and 32 (empty in the base), prompt template `afm_cider_v2`. The `base.generic` image is 1.3 GB, i.e. ≈ 2 bits per weight. The 3B embedding asset of §17 declares `embedding_dim` 2048 and therefore serves this backbone rather than the 1536-wide sparse one.

9.4. The shipped 3B embedding asset.

Property	Value
Asset	UC_FM_LANGUAGE_INSTRUCT_3B_EMBEDDINGS_..._H16G_IFP_Cryptex.dmg
Container	read-only cryptex, mountable without privilege
Declared shape	vocab_size 262144, embedding_dim 2048
Payload file	model.odixpackage/main-h16g.odix, 268,442,456 B
Arithmetic	262144×2048 at 4 bits = 268,435,456 B + 7000 B header
Header	odix flatbuffer; strings include \$load_embeddings; NSArray descriptor [262144, 1, 2048]
Encoding	linear signed int4; histogram symmetric about zero, code 8 unused
Codebook / scale	none of either
Row stride	1024 B, token ids in identity order
Padding	final 147,456 B = exactly 144 rows, one repeated pattern
Lane structure	8 lanes; 332 duplicate-row groups, every large one 100% pure in id mod 8
Status	per-token vectors not recovered

The 144-row padding figure is corroborated independently: the draft model’s captured logit vector carries exactly 144 masked entries, giving a live vocabulary of 262,000 in both models.

9.5. Weight codec, common to all three. Four-bit LUT palettization: a 4-bit index selects from a 16-entry fp16 codebook, scaled by a per-block fp16 factor, stored under an Apple Neural Engine tile permutation. For the 3B the de-swizzle is $W = v.\text{reshape}(C_o/8, C_i/128, 128, 8).\text{transpose}(0, 3, 1, 2)$ with the scale applied in raw index order *before* de-swizzling. For the draft model the equivalent transform is the bank z-order given above. Under a scale-decontaminated low-rank structure test, genuine weights score $R \approx 9$ and noise $R \approx 1$; the decode scores $R = 13\text{--}69$.

10. CONTAINER FORMATS

The on-disk stack is summarized in Figure 6.

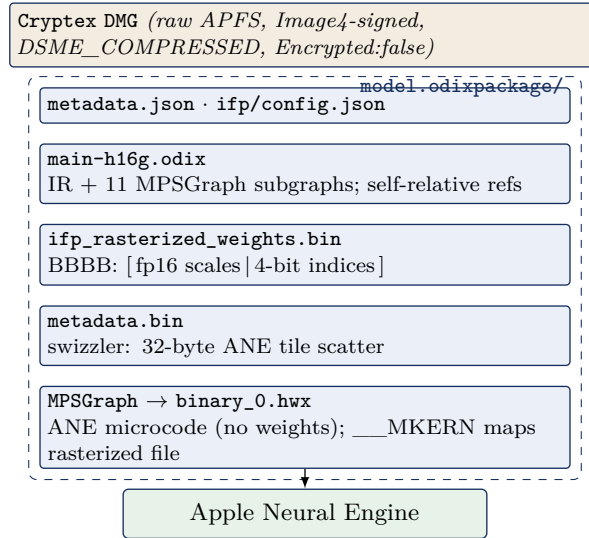


Figure 6. The FM_GenerativeModels on-disk container stack for the IFP model. The dashed box is model.odixpackage/; execution is dispatched to the ANE.

Cryptex DMG. Raw APFS image, DSME_COMPRESSED; contains model.odixpackage/, metadata.json, a nominal System/Library. **BBBB rasterized container** (ifp_rasterized_weights.bin): magic BBBB, version 0x017c0100, tag 0x0600002c, then a u64 segment-offset table—fp16 scales

0x60–0x1054000; 4-bit indices 0x1078000–0x125e8000. **Swizzler** (`metadata.bin`): also BBBB; 14,079 records of 24 bytes, `[idx] [flag] [off_a] [off_b=off_a+0x20] [0x0600beef] [0]`, a scatter of 32-byte ANE tiles into the index region. **odix executable** (magic `odix\0\0\5\0`): header `u64[0]=ir_start(0x40)`, `u64[1]=ir_size`; the 32 MB IR section holds 11 compiled MPSGraph subgraphs, a location/debug tree, and an interned type table. **ANE .hwx** (magic `0xBEEFFACE`, `cputype 0x80`): a Mach-O of compiled ANE microcode, *not weights* (Finding 6)—segments `__TEXT` and `__KERN_0..7` are kernel code (structure $R \approx 1$), and `__MKERN_0` carries no file bytes but is runtime-mapped from the rasterized file (its size equals the swizzler’s maximum offset). **MPSGraph package**. `specialized_model_0.mpsgraph` is MLIR bytecode (MLIR22.0.0) with `mps.fullyPlacedOnANE`.

Finding 26. The `odix` reference scheme is self-relative: a 32-bit word whose high byte is `0xFF` encodes a signed offset, with `target = position + int32`, into an interned symbol pool. Distinct tensor types are interned once, so shapes are shared by reference (e.g. “1536” occurs only 26 times in 32 MB).

11. TOKENIZER

A standalone 8.6 MB SentencePiece-family tokenizer. Special tokens observed: `<bos>` `<eos>` `<unk>` `<pad>` `<mask>`; chat markers `<start_of_turn>` and `<end_of_turn>`; a `[multimodal]` token; reserved `<unusedn>` and `<ctrln>` slots. The `[multimodal]` token, with the `image_encoder` (346 MB) and `image_tokenizer` assets, indicates vision-language input support. Estimated vocabulary 152,064.

12. ADAPTERS AND TASK SPECIALIZATION

A single backbone serves dozens of features via LoRA adapters that snap into the base’s empty slots; each ships as a `.generic` adapter plus a small `.draft`. **generic** (38–62 MB) speculative-decoding draft. **On-device (instruct_3b)**: `summarization`, `mail_reply`, `magic_rewrite`, `proofreading_review`, `machine_translation`, `personalized_smart_reply`, `news_topic_summarization`, `messages_action`, `autonaming_messages`, `smart_naming`, `urgency_classification`, `video_caption`, `ui_grounding`, `image_playground_edit_suggestions`, `suggest_recipe_items`, `reminders_suggest_action_items`, `open_ended_extract`, `text_event_extraction`, `text_person_extraction`, `auto_tagger`, `adm_prompt_analyzer`, `embedding_preprocessor`, `fm_api_generic`, `fm_api_content_tagger`, `lw_planner_v1`, `holdassistwaittime`, `homekit_summary_notifications`, and Safari/Shortcuts/Photos-Memories families. **Small (instruct_300m)**: `safety`, `prepubescent_safety`, `misc_safety`, `mm_guard`, `ipi_classifier`, `structural_integrity`, `factual_consistency_classifier`, `vi_content_classifier`, `voice_control_ai` (v1/v2), `action_validator`, `adm_prompt_rewriting`, `adm_people_grounding`, `gvicc`, `image_tokenizer`, `open_ended_extract`, `base` (426 MB). **instruct_9m**: a 9 M-parameter text-event-extraction classifier. **Server / PCC (instruct_server_v1)**: `text_summarizer`, `takeaways/tables/bullets_transform`, `professional/friendly/concise_tone`, `open_ended_tone` (+ `query-response` v1/v2), `mail_reply_long_form_{basic,rewrite}`, `mail_reply_qa`, `magic_rewrite`, `generative_shortcuts`, `llm_siri_pcc`, `lw_planner_v1`, `fitness_workout_voice`, `reminders_auto_categorized_list`, `photos_memories_*`, `stx_multimodal`, `shortcuts_ask_afm_action` (v1/v2); **instruct_server_small**: `safety_nsfw`, `model_abuse_guardrail`. **Translation (gm.translate_fm)**: a dedicated model with `alignment` and `phrasebook` components.

13. REVERSE-ENGINEERING METHODOLOGY

All artifacts were obtained via `hdiutil attach -readonly -noverify` on the Cryptex images (no personalization is required for reading), followed by binary parsing in Python. Architecture was recovered from `config.json/metadata.json` and from `tamm-framework` tensor names

inside `main-h16g.odix` (e.g. `_wrapped_model_layers_..._palettized_indices_raw`, `_wrapped_model_output_norm_weight`). The quantization scheme (Finding 1) was inferred from value-distribution analysis of the two BBBB regions; the reference scheme (Finding 26) from resolving the `0xFF`-tagged words; and the expert fan-out (Finding 2) from the size budget; the per-constant layer/role semantics were read from the MLIR22.0.0 MPSGraph bytecode location tree. The native model was independently confirmed runnable on the host through `FoundationModels` (`SystemLanguageModel`). Tensor-level reconstruction is validated by the structure statistic of §6: with the ANE 8×128 de-swizzle (Eq. 1) the decode scores $R = 13\text{--}69$ against a genuine-weight reference of ≈ 9 , and the full export reconciles to 100.00% of the physical index region with zero non-finite tensors (Table 6). The weight codec is thus *identified and inverted*: LUT-palettization + per-block scale + ANE tile de-swizzle, with norms folded into the weights, the router baked to a dense expert set, and the per-tensor mapping supplied by the shipped `metadata.bin` swizzler in lieu of the compile-time `ifp_constant_table_ifp1_r48.json`. A complete, named `state_dict` is produced.

14. COMPLETE ASSET INVENTORY

Table 8 lists all 118 assets with version and disk-image size in megabytes (blank denotes a metadata-only override with no payload).

Table 8. Full `FM_GenerativeModels` asset inventory (118 assets).

Bundle name	Version	MB
<code>fm.generativemodels.uaf.metadata</code>	4.46.3	
<code>fm.language.instruct_300m.action_validator.generic</code>	14.1.0	
<code>fm.language.instruct_300m.adm_people_grounding.generic</code>	13.10002.0	
<code>fm.language.instruct_300m.adm_prompt_rewriting.draft.generic</code>	13.10002.81607	40
<code>fm.language.instruct_300m.adm_prompt_rewriting.generic</code>	13.10002.0	
<code>fm.language.instruct_300m.base.generic</code>	14.0.81607	447
<code>fm.language.instruct_300m.factual_consistency_classifier.generic</code>	14.0.0	
<code>fm.language.instruct_300m.gvicc.generic</code>	14.0.0	
<code>fm.language.instruct_300m.image_tokenizer.generic</code>	14.1.81607	359
<code>fm.language.instruct_300m.ipi_classifier.generic</code>	14.1.0	
<code>fm.language.instruct_300m.misc_safety.generic</code>	14.0.0	
<code>fm.language.instruct_300m.mm_guard.generic</code>	14.0.0	
<code>fm.language.instruct_300m.open_ended_extract.draft.generic</code>	14.2.81607	65
<code>fm.language.instruct_300m.open_ended_extract.generic</code>	14.2.0	
<code>fm.language.instruct_300m.prepubescent_safety.generic</code>	14.0.0	
<code>fm.language.instruct_300m.safety.generic</code>	14.1.0	
<code>fm.language.instruct_300m.structural_integrity.generic</code>	14.0.0	
<code>fm.language.instruct_300m.tokenizer.generic</code>	14.0.0	
<code>fm.language.instruct_300m.vi_content_classifier.generic</code>	13.10004.0	
<code>fm.language.instruct_300m.voice_control_ai.generic</code>	14.1.0	
<code>fm.language.instruct_300m.voice_control_ai_v2.generic</code>	13.10001.0	
<code>fm.language.instruct_300m.voice_control_ai_v2.image_tokenizer_adapter.generic</code>	13.10001.0	
<code>fm.language.instruct_3b.adm_prompt_analyzer.generic</code>	14.0.0	
<code>fm.language.instruct_3b.auto_tagger.generic</code>	12.0.0	
<code>fm.language.instruct_3b.autonaming_messages.draft.generic</code>	14.1.81607	65
<code>fm.language.instruct_3b.autonaming_messages.generic</code>	14.1.0	
<code>fm.language.instruct_3b.base.generic</code>	14.0.81607	1380
<code>fm.language.instruct_3b.base.generic_sparse</code>	14.3.81614	5828
<code>fm.language.instruct_3b.embedding_preprocessor.generic</code>	14.2.0	

continued on next page

Table 8, continued

Bundle name	Version	MB
fm.language.instruct_3b.fm_api_content_tagger.adapter_metadata_override.generic	14.1.0	
fm.language.instruct_3b.fm_api_generic.draft.generic	14.2.81607	61
fm.language.instruct_3b.fm_api_generic.generic	14.2.0	
fm.language.instruct_3b.holdassistwaittime.adapter_metadata_override.generic	14.0.0	
fm.language.instruct_3b.homekit_summary_notifications.adapter_metadata_override.generic	14.2.0	
fm.language.instruct_3b.image_encoder.generic	14.1.81607	363
fm.language.instruct_3b.image_encoder.generic_sparse	14.3.81614	361
fm.language.instruct_3b.image_playground_edit_suggestions.generic	14.1.0	
fm.language.instruct_3b.lw_planner_v1.draft.generic	14.1.81607	55
fm.language.instruct_3b.lw_planner_v1.generic	14.1.0	
fm.language.instruct_3b.machine_translation.draft.generic	14.0.81607	55
fm.language.instruct_3b.machine_translation.generic	14.0.0	
fm.language.instruct_3b.mail_reply.draft.generic	14.1.81607	65
fm.language.instruct_3b.mail_reply.generic	14.1.0	
fm.language.instruct_3b.messages_action.draft.generic	14.1.81607	65
fm.language.instruct_3b.messages_action.generic	14.1.0	
fm.language.instruct_3b.news_topic_summarization.draft.generic	14.3.81607	65
fm.language.instruct_3b.news_topic_summarization.generic	14.3.0	
fm.language.instruct_3b.open_ended_extract.draft.generic	14.2.81607	65
fm.language.instruct_3b.open_ended_extract.generic	14.2.0	
fm.language.instruct_3b.personalized_smart_reply.draft.generic	14.2.81607	65
fm.language.instruct_3b.personalized_smart_reply.generic	14.2.0	
fm.language.instruct_3b.photos_library_understanding_mm.generic	14.3.0	
fm.language.instruct_3b.photos_memories_asset_curation_outlier.draft.generic	14.1.81607	65
fm.language.instruct_3b.photos_memories_asset_curation_outlier.generic	14.1.0	
fm.language.instruct_3b.photos_memories_title.adapter_metadata_override.generic	14.1.0	
fm.language.instruct_3b.photos_memories_title.draft.generic	13.10003.81607	40
fm.language.instruct_3b.photos_memories_title.generic	13.10003.0	
fm.language.instruct_3b.proofreading_review.draft.generic	14.1.81607	65
fm.language.instruct_3b.proofreading_review.generic	14.1.0	
fm.language.instruct_3b.reminders_suggest_action_items.draft.generic	14.1.81607	65
fm.language.instruct_3b.reminders_suggest_action_items.generic	14.1.0	
fm.language.instruct_3b.safari_magic_extensions_app_store_search_terms.adapter_metadata_override.generic	14.2.0	
fm.language.instruct_3b.safari_notify_me_when_suggestions.adapter_metadata_override.generic	14.2.0	
fm.language.instruct_3b.safari_tab_group_topic.adapter_metadata_override.generic	14.3.0	
fm.language.instruct_3b.shortcuts_ask_afm_action_3b.adapter_metadata_override.generic	14.0.0	
fm.language.instruct_3b.shortcuts_ask_afm_action_3b_v2.draft.generic	10.34.81607	55
fm.language.instruct_3b.shortcuts_ask_afm_action_3b_v2.generic	10.34.0	
fm.language.instruct_3b.smart_naming.generic	14.1.0	
fm.language.instruct_3b.suggest_recipe_items.draft.generic	14.1.81607	65
fm.language.instruct_3b.suggest_recipe_items.generic	14.1.0	
fm.language.instruct_3b.summarization.draft.generic	14.5.81607	61
fm.language.instruct_3b.summarization.generic	14.5.0	
fm.language.instruct_3b.text_event_extraction.draft.generic	13.10003.81607	40
fm.language.instruct_3b.text_event_extraction.generic	13.10003.0	

continued on next page

Table 8, continued

Bundle name	Version	MB
fm.language.instruct_3b.text_person_extraction.draft.generic	14.1.81607	65
fm.language.instruct_3b.text_person_extraction.generic	14.1.0	
fm.language.instruct_3b.tokenizer.generic	14.0.0	
fm.language.instruct_3b.tokenizer.generic_sparse	14.3.0	
fm.language.instruct_3b.ui_grounding.generic	14.0.0	
fm.language.instruct_3b.urgency_classification.generic	14.2.0	13.10000.0
fm.language.instruct_3b.video_caption.generic	13.10000.0	
fm.language.instruct_3b.voice.generic_sparse	14.1.0	
fm.language.instruct_9m.text_event_extraction_classifier.base.generic	1.1.81607	
fm.language.instruct_9m.text_event_extraction_classifier.tokenizer.generic	1.0.0	
gm.server.instruct_server_small.model_abuse_guardrail.generic	13.10006.0	113
gm.server.instruct_server_small.safety_nsfw.generic	13.10008.0	
gm.server.instruct_server_v1.bullets_transform.generic	12.4.0	
gm.server.instruct_server_v1.concise_tone.generic	11.0.0	
gm.server.instruct_server_v1.fitness_workout_voice.generic	12.51.0	
gm.server.instruct_server_v1.friendly_tone.generic	12.0.0	12.16.0
gm.server.instruct_server_v1.generative_shortcuts.generic	12.16.0	
gm.server.instruct_server_v1.llm_siri_pcc.generic	11.33.0	
gm.server.instruct_server_v1.lw_planner_v1.generic	12.2.0	
gm.server.instruct_server_v1.magic_rewrite.generic	11.0.0	
gm.server.instruct_server_v1.mail_reply_long_form_basic.generic	12.0.0	12.0.0
gm.server.instruct_server_v1.mail_reply_long_form_rewrite.generic	12.0.0	
gm.server.instruct_server_v1.mail_reply_qa.generic	12.0.0	
gm.server.instruct_server_v1.open_ended_schema.generic	12.10.0	
gm.server.instruct_server_v1.open_ended_tone.generic	12.24.0	
gm.server.instruct_server_v1.open_ended_tone_query_response.generic	12.0.0	12.0.0
gm.server.instruct_server_v1.open_ended_tone_query_response_v2.generic	12.0.0	
gm.server.instruct_server_v1.photos_memories_asset_curation_v2.generic	12.0.0	
gm.server.instruct_server_v1.photos_memories_global_traits_v2.generic	12.0.0	
gm.server.instruct_server_v1.photos_memories_global_traits_v3.generic	12.0.0	
gm.server.instruct_server_v1.photos_memories_query_understanding_v3.generic	12.4.0	12.0.0
gm.server.instruct_server_v1.photos_memories_storyteller_v2.generic	12.0.0	
gm.server.instruct_server_v1.professional_tone.generic	12.0.0	
gm.server.instruct_server_v1.reminders_auto_categorized_list.generic	12.0.0	
gm.server.instruct_server_v1.shortcuts_ask_afm_action.generic	11.19.0	
gm.server.instruct_server_v1.shortcuts_ask_afm_action_v2.generic	12.54.0	12.20.0
gm.server.instruct_server_v1.stx_multimodal.generic	12.20.0	
gm.server.instruct_server_v1.tables_transform.generic	12.0.0	
gm.server.instruct_server_v1.takeaways_transform.generic	12.0.0	
gm.server.instruct_server_v1.text_summarizer.generic	12.0.0	
gm.translate_fm.machine_translation.alignment.generic	14.1.0	14.1.0
gm.translate_fm.machine_translation.phrasebook.generic	14.1.0	
mm_guard.output.configuration.generic	0.0.2	0.1.6
safety.output.configuration.generic	0.1.6	

APPENDIX A. ODIX IR GRAMMAR

The odix IR is a length-prefixed section stream. Strings and tensor *types* are interned once in a symbol pool and shared by reference. A reference is a 32-bit word whose high byte is 0xFF,

encoding a self-relative signed offset $t = p + \text{int32}(w)$ from the word position p . Attribute records in the location/debug tree take the form $\langle \text{key-ref} \rangle \langle \text{len:u32} \rangle \langle \text{payload} \rangle$ with interned keys `name`, `location_type`, `named_child_loc`, `op_id`, `line`, `filename`, `column`, `metadata`. Because types are interned, a dimension such as 1536 occurs only ≈ 26 times across the entire 32 MB IR. The compile-time name-to-offset table (`ifp_constant_table_ifp1_r48.json`) is not shipped, but it is not needed: its runtime equivalent is the `metadata.bin` BBBB swizzler (Appendix B), and the MPSGraph bytecode location tree tags each `ifp_constant` with its layer/role, so a named export is produced without it (§6).

APPENDIX B. BBBB CONTAINER LAYOUT

Both `ifp_rasterized_weights.bin` and the swizzler `metadata.bin` share the BBBB magic. The rasterized header is BBBB, version `0x017c0100`, tag `0x0600002c`, count, then a u64 segment-offset table. The swizzler body is a sequence of 24-byte records `[idx][flag][off_a][off_b][marker=0x0600beef][0]` with `off_b = off_a + 0x20`, i.e. a scatter map of 32-byte Apple Neural Engine tiles. Region spans: scales `[0x60,0x1054000]`; 4-bit indices `[0x1078000,0x125e80000]`.

REFERENCES

- [1] Apple, *Introducing Apple’s On-Device and Server Foundation Models*, Apple Machine Learning Research, 2024.
- [2] S. Mehta et al., *OpenELM: An Efficient Language Model Family with Open Training and Inference Framework*, arXiv:2404.14619, 2024.
- [3] W. Fedus, B. Zoph, N. Shazeer, *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*, JMLR, 2022.
- [4] N. Shazeer et al., *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*, ICLR, 2017.
- [5] J. Ainslie et al., *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*, EMNLP, 2023.
- [6] J. Su et al., *RoFormer: Enhanced Transformer with Rotary Position Embedding*, arXiv:2104.09864, 2021.
- [7] B. Zhang, R. Sennrich, *Root Mean Square Layer Normalization*, NeurIPS, 2019.
- [8] N. Shazeer, *GLU Variants Improve Transformer*, arXiv:2002.05202, 2020.
- [9] G. Gerganov et al., *GGUF / llama.cpp: A file format and runtime for quantized transformer inference*, <https://github.com/ggml-org/llama.cpp>, 2023-.